

Project Overview

Features, setup and operations guide

Research Assistant - local-first multi-agent research workspace
Generated from README.md

Research Assistant

Research Assistant is a local-first, privacy-preserving research workspace for finding scholarly sources, pinning them into persistent research spaces, ingesting their text into local embeddings, and asking citation-verified questions against that evidence. The implemented application combines multi-source academic discovery, optional web capture, a paper reader with annotations, memory, comparison reports, and a hand-written peer-review verification loop that makes every grounded answer auditable.

Key features implemented

- Multi-source scholarly discovery through arXiv, OpenAlex, Crossref, PubMed, Semantic Scholar, and CORE adapters.
- Domain tags such as `#ml`, `#nlp`, `#medical`, and `#physics` route search toward relevant sources, arXiv categories, and query terms.
- Optional web search via DuckDuckGo Lite by default, with Brave, Tavily, and SearxNG provider hooks.
- Google Scholar deep links in the UI. The backend deliberately does not fetch Google Scholar.
- Paper pinning, PDF or abstract ingestion, page-aware chunking, local Ollama embeddings, and persistent chunk rows.
- Hybrid retrieval with dense vector search, sparse keyword search, RRF fusion, heuristic reranking, MMR diversification, and context budgeting.
- Phase 3 peer-review verification loop with draft generation, reviewer verdicts, policy routing, best-effort finalization, abstention, and visible verification trail.
- Compare and gap analysis over pinned papers, backed by paper profiles and the same verification loop.
- Auto notes, manual notes, anchored highlights, paper reader, and editable annotation cards.
- Persistent research spaces with archive, unarchive, duplicate, delete, turn history, memory, and content search.

Screenshots

Area	Placeholder	Caption
Dashboard	<code>! [Dashboard screenshot](docs/images/dashboard-placeholder.png)</code>	Research spaces and backend health.
Discovery	<code>! [Discovery screenshot](docs/images/discovery-placeholder.png)</code>	Domain-tag search, source health, web results, and pinning.
Chat	<code>! [Chat screenshot](docs/images/chat-placeholder.png)</code>	Verified local answers with citation popovers and trail disclosure.
Paper reader	<code>! [Reader screenshot](docs/images/reader-placeholder.png)</code>	Page-aware chunks, highlights, and anchored notes.
Compare	<code>! [Compare screenshot](docs/images/compare-placeholder.png)</code>	Matrix, commonalities, contradictions, gaps, and confidence.

The image paths are placeholders only. No screenshot files are included.

Quickstart on Windows PowerShell

Prerequisites

- Python 3.13.
- Node.js 22.

- Ollama running locally on `http://localhost:11434`.
- Ollama models pulled locally:

```
ollama pull llama3.2:3b
ollama pull gemma3
ollama pull nomic-embed-text
```

The checked-in backend defaults still name older `llama3.1:8b` values in `backend/.env.example`, but the delivered local configuration uses `llama3.2:3b`, `gemma3:latest`, and `nomic-embed-text`.

Backend

```
cd C:\Users\devikrishna.u\projects\research-assistant\backend
py -3.13 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e ".[test]"
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8321 --reload
```

Backend URL: `http://localhost:8321`. Live OpenAPI: `http://localhost:8321/openapi.json`.

Frontend

```
cd C:\Users\devikrishna.u\projects\research-assistant\frontend
npm install
npm run dev
```

Frontend URL: `http://localhost:3000`.

Environment variables that matter

Place backend values in `backend/.env`.

Variable	Purpose	Delivered local value or default
<code>DATABASE_URL</code>	SQLAlchemy database URL.	<code>sqlite+aiosqlite:///./data/dev.db</code>
<code>OLLAMA_BASE_URL</code>	Local Ollama host.	<code>http://localhost:11434</code>
<code>OLLAMA_MODEL_SMALL</code>	Query rewriting and light work.	<code>llama3.2:3b</code>
<code>OLLAMA_MODEL_MEDIUM</code>	Answer generation.	<code>llama3.2:3b</code>
<code>OLLAMA_MODEL_LARGE</code>	Late reviewer and heavier synthesis.	<code>gemma3:latest</code>
<code>OLLAMA_MODEL_VERIFY</code>	Peer reviewer.	<code>gemma3:latest</code>
<code>OLLAMA_MODEL_EMBED</code>	Embeddings.	<code>nomic-embed-text</code>
<code>OLLAMA_TIMEOUT_S</code>	Ollama HTTP timeout.	<code>120</code> in local <code>.env</code> ; config default is <code>15</code> .
<code>SOURCE_USER_AGENT</code>	User-Agent sent to public APIs.	Set a polite value with contact info, for example <code>research-assistant/0.1 (mailto:you@example.com)</code> .
<code>SEMANTIC_SCHOLAR_API_KEY</code>	Required before the Semantic Scholar adapter is used.	Optional but recommended.

Variable	Purpose	Delivered local value or default
CORE_API_KEY	Enables CORE results.	Optional.
PUBMED_API_KEY	Raises PubMed E-utilities limits.	Optional.
WEB_SEARCH_PROVIDER	Web provider.	duckduckgo; alternatives: brave, tavily, searxng.
BRAVE_API_KEY, TAVILY_API_KEY, SEARXNG_URL	Provider-specific web search settings.	Optional.
SOURCE_TIMEOUT_S	Hard per-source discovery deadline.	8.0.
LOOP_MAX_ITERATIONS	Verification loop iteration cap.	3.
LOOP_WALL_CLOCK_CAP_MS	Verification wall-clock cap.	120000 in local .env; config default is 20000.
LOOP_ACCEPTANCE_THRESHOLD	Reviewer score needed for approval.	0.90.

How it works

A user asks a question inside a research space. The backend rewrites referential follow-ups into a short standalone retrieval query, retrieves pinned paper chunks with dense and sparse search, packs a bounded evidence set, drafts an answer with citations, and sends the draft to a separate peer reviewer. Loop policies validate claim coverage and citation IDs, route to regenerate or re-retrieve when needed, and either accept, return a best-effort answer with low-confidence warning, or abstain. Citations and every verification iteration are persisted and displayed in the UI.

Tech stack

Layer	Implemented technology
Frontend	Next.js, React, TypeScript, Tailwind CSS, lucide-react, react-markdown
Backend API	FastAPI, Pydantic, SQLAlchemy asyncio, aiosqlite
Local inference	Ollama chat and embedding APIs
Discovery	httpx source adapters for arXiv, OpenAlex, Crossref, PubMed, Semantic Scholar, CORE, and web search providers
Retrieval	In-process vector store plus SQLite-persisted embeddings, TF-IDF-like sparse search, RRF, rerank, MMR
Persistence	SQLite development database at backend\data\dev.db
Tests	pytest and pytest-asyncio for backend; TypeScript, ESLint, and Next build for frontend

Project layout

```
research-assistant\  
  backend\  
    app\  
      agents\  
      api\v1\  
      core\  
      llm\  
      models\  
      retrieval\  
      schemas\  
      services\  
      sources\  
      verification\  
      vectorstore\  
    data\dev.db  
    tests\  
  frontend\  
    app\  
    components\  
    lib\  
    docs\  
      LLM-facing answer, reviewer, compare, gap, note, memory, and query agents  
      FastAPI route modules  
      settings, database dependencies, startup migrations, exceptions  
      Ollama client and model router  
      SQLAlchemy tables  
      dense, sparse, fusion, rerank, MMR, context packing, query rewriting  
      Pydantic request and response contracts  
      orchestration, spaces, papers, notes, memory, compare, health  
      scholarly and web source adapters, taxonomy, registry, dedupe  
      loop driver, policies, scoring, trail persistence  
      in-memory dev vector store and Qdrant health stub  
      SQLite development database  
      85 backend tests  
      Next.js app routes  
      dashboard, workspace, reader, notes, compare, markdown, badges  
      API client, shared types, utilities  
      design-phase docs plus current HLD and LLD
```

Testing and validation commands

Backend test suite currently contains 85 tests.

```
cd C:\Users\devikrishna.u\projects\research-assistant\backend  
.\venv\Scripts\python.exe -m pytest -q
```

Frontend validation commands:

```
cd C:\Users\devikrishna.u\projects\research-assistant\frontend  
npm run typecheck  
npm run lint  
npm run build
```

Known limitations and caveats

- Small local models can be slow or conservative, and may abstain when evidence is thin.
- Public scholarly APIs have rate limits. Semantic Scholar is intentionally skipped unless `SEMANTIC_SCHOLAR_API_KEY` is set.
- SQLite is the default development database. There is no production Postgres migration path wired yet.
- Auth and multi-tenancy are not implemented, despite a `users` table existing.
- Alembic is not wired. Startup uses `create_all` plus a small idempotent migration helper for note anchor columns and citation metadata.
- Celery and LangGraph were not adopted. The verification loop is a plain async Python driver.
- Qdrant and GROBID are configured or stubbed but not required. The delivered path stores embeddings in SQLite JSON and an in-memory vector store, and parses PDFs with `pypdf`.
- Highlight rendering clips overlapping ranges. Fully nested highlights can be hidden in the reader, though notes remain accessible from the Notes tab.
- Exact-paper lookup methods exist on the adapter interface but are not implemented as live routes.

Roadmap

1. Add authentication and proper workspace ownership.
2. Replace startup migrations with Alembic.
3. Move embeddings to a real Qdrant collection or another durable vector index.

4. Add a production database profile, likely Postgres.
5. Add a worker queue for long ingestion and comparison jobs if concurrent use grows.
6. Improve PDF structure extraction, potentially with GROBID, while preserving the current SSRF and size guards.
7. Add screenshot assets and user-facing docs once UI screenshots are captured.
8. Add evaluation datasets for verification accuracy, latency, and abstention behavior.